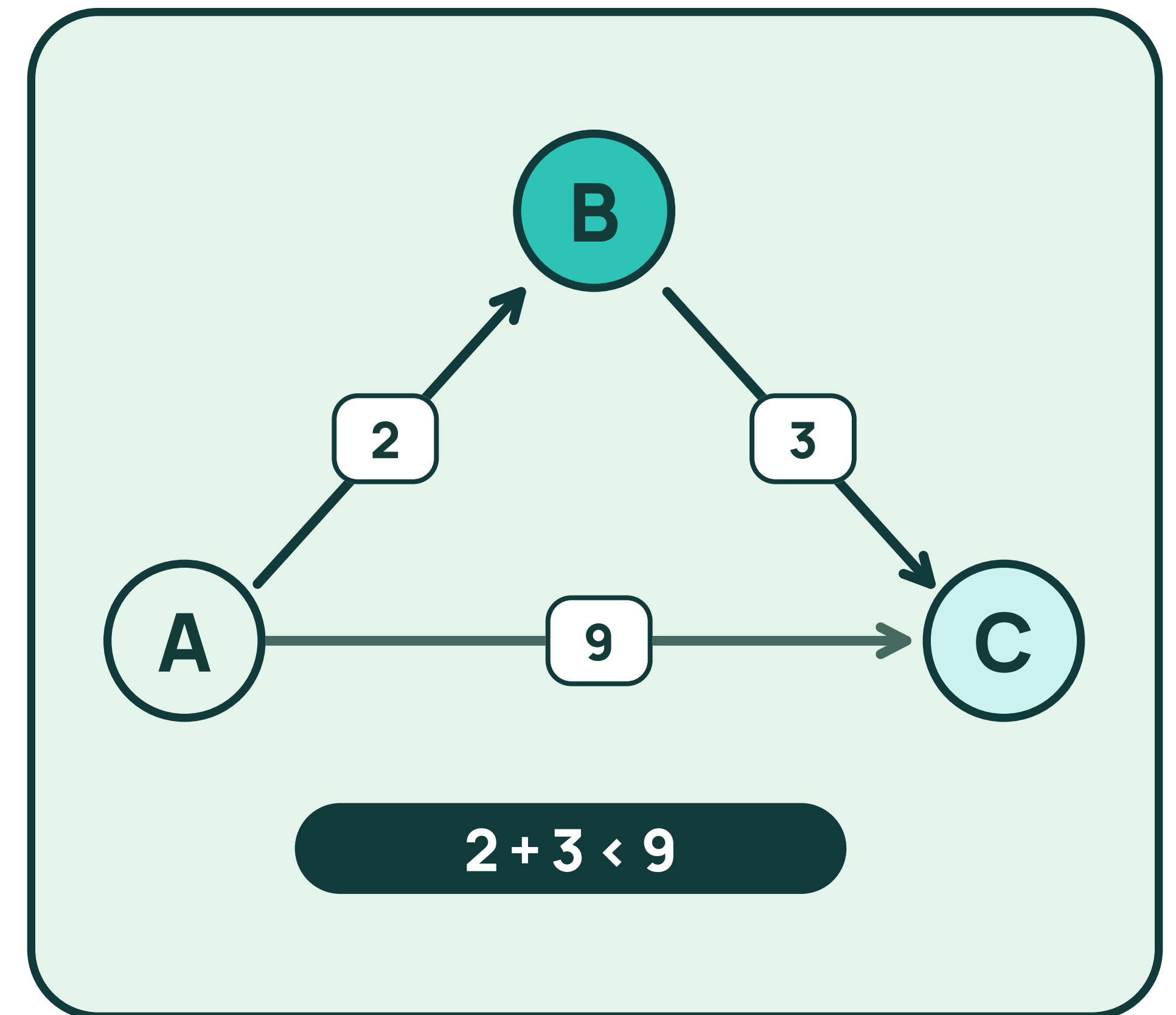


Todos os caminhos. Uma ideia.

Floyd-Warshall

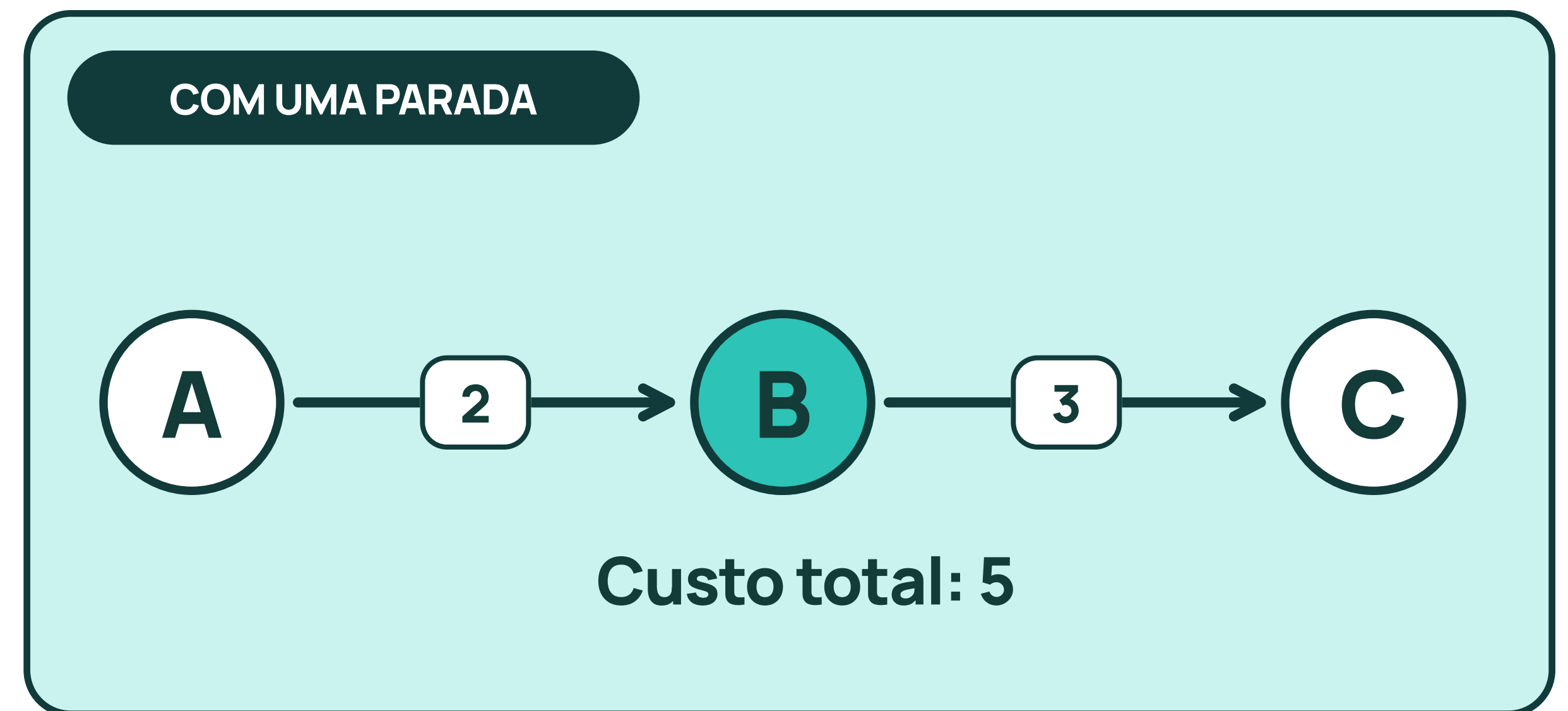
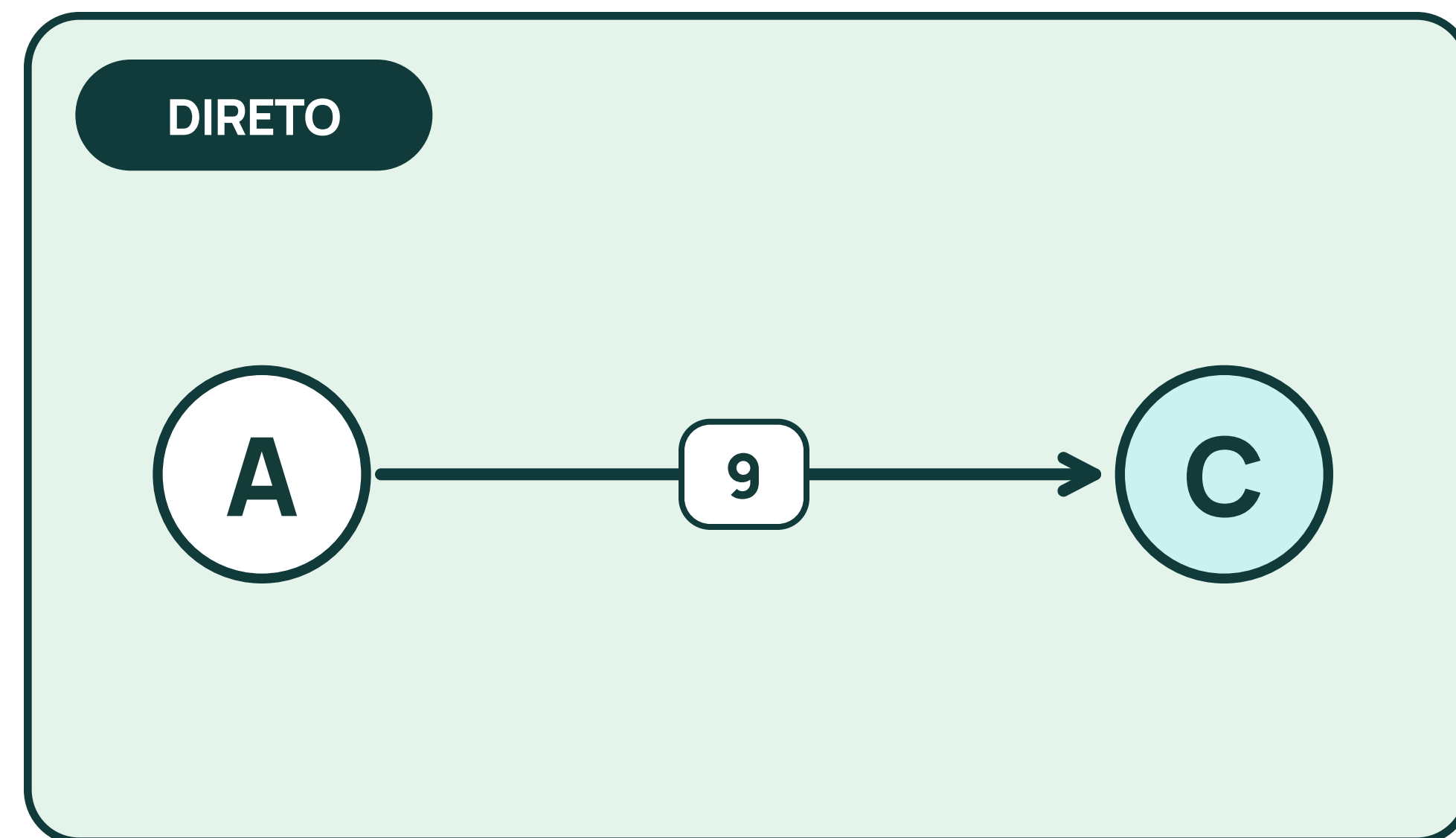
Caminhos mínimos entre todos os pares.

FLOYD-WARSHALL / Amanda Dias & Camilla Borchhardt



Menor custo entre todos os pares.

Entrada: grafo dirigido com pesos. Saída: $d[i][j]$ para todo par (i, j) . Restrição: sem ciclo negativo.



Está em P: $O(V^3)$ é tempo polinomial.

Rotas, redes e jogos usam Floyd-Warshall.



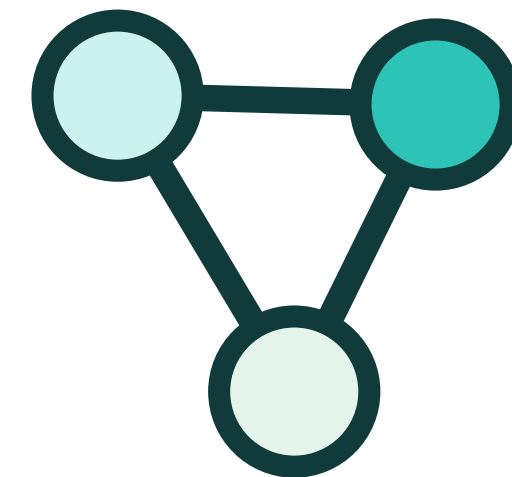
Rotas

Cidades ligadas por estradas com custo de viagem.



Redes

Computadores e roteadores; menor custo entre todos.

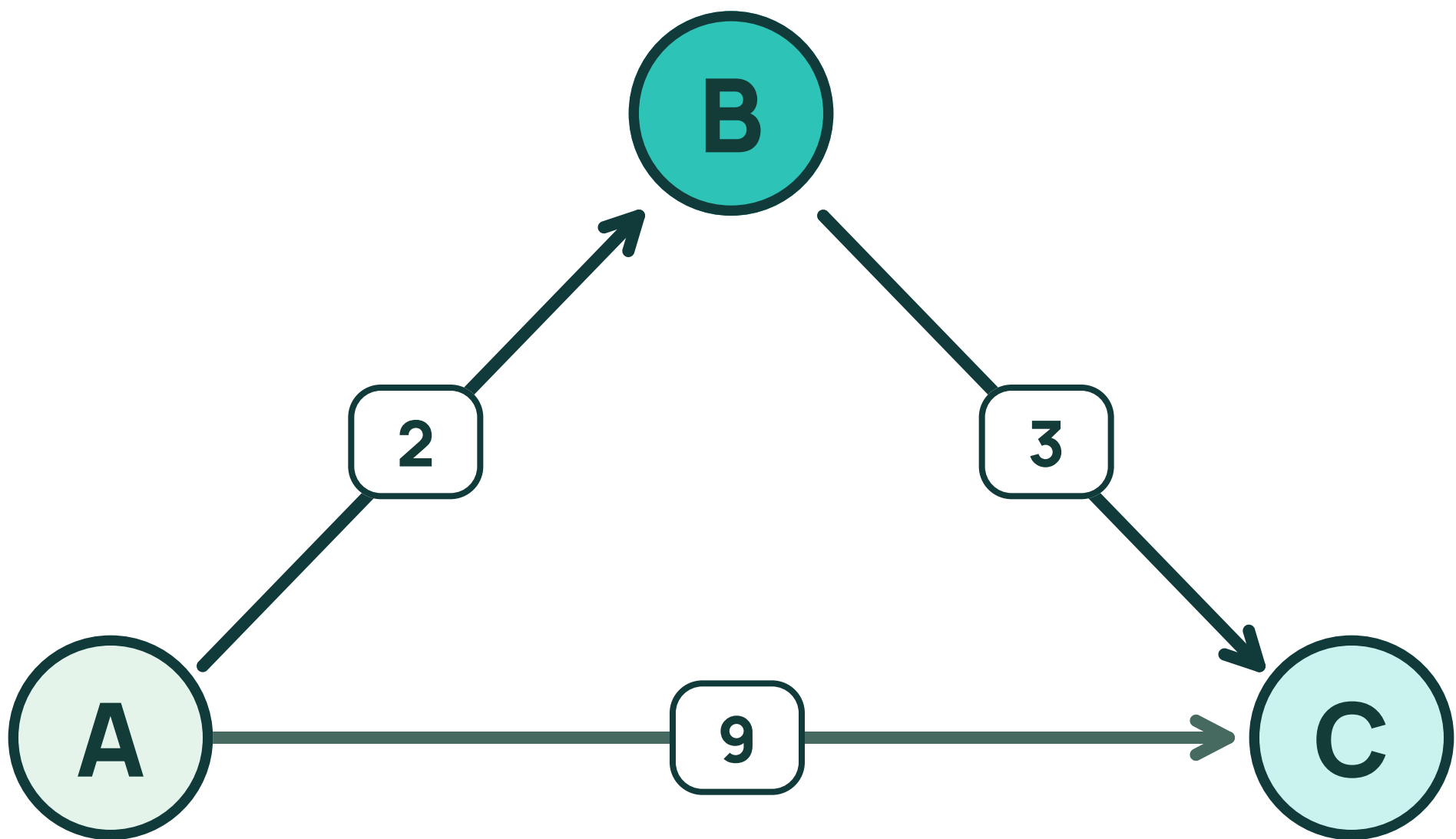


Jogos

Mapas com regiões e caminhos entre elas.

O grafo vira uma matriz de distâncias.

Linha = origem, coluna = destino. 0 na diagonal, peso onde há aresta, ∞ onde não há.



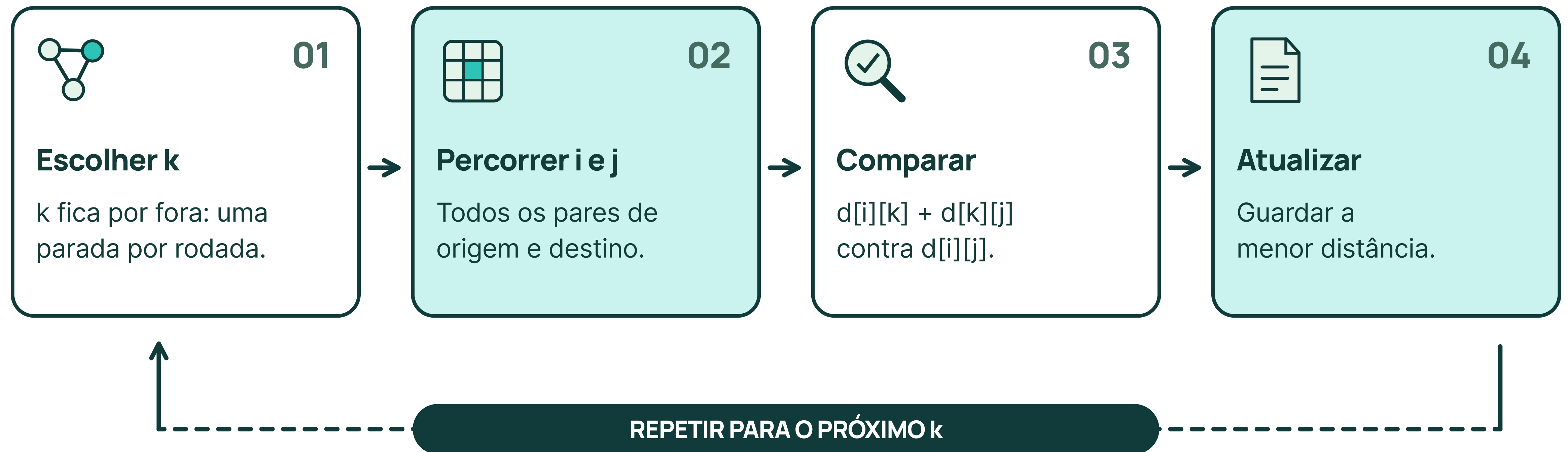
	A	B	C
A	0	2	9
B	∞	0	3
C	∞	∞	0

GRAFO

MATRIZ DE DISTÂNCIAS

A REGRA: $d[i][j] = \min(d[i][j], d[i][k] + d[k][j])$

O algoritmo como um percurso.



RODADAS $k = B$ E $k = C$

Duas rodadas mudam a linha A.

grafo_exemplo.csv: $A \rightarrow B$ 3, $B \rightarrow C$ 2, $C \rightarrow D$ 1, $A \rightarrow D$ 10. Matriz após as duas rodadas.

	A	B	C	D
A	0	3	5	6
B	∞	0	2	3
C	∞	∞	0	1
D	∞	∞	∞	0

$A \rightarrow B \rightarrow C$ (rodada $k = B$)

$3 + 2 = 5$ | antes: ∞

$A \rightarrow C \rightarrow D$ (rodada $k = C$)

$5 + 1 = 6$ | antes: 10

A distância não diz por onde ir.

Guardamos também o próximo vértice de cada par: `next[i][j]`.

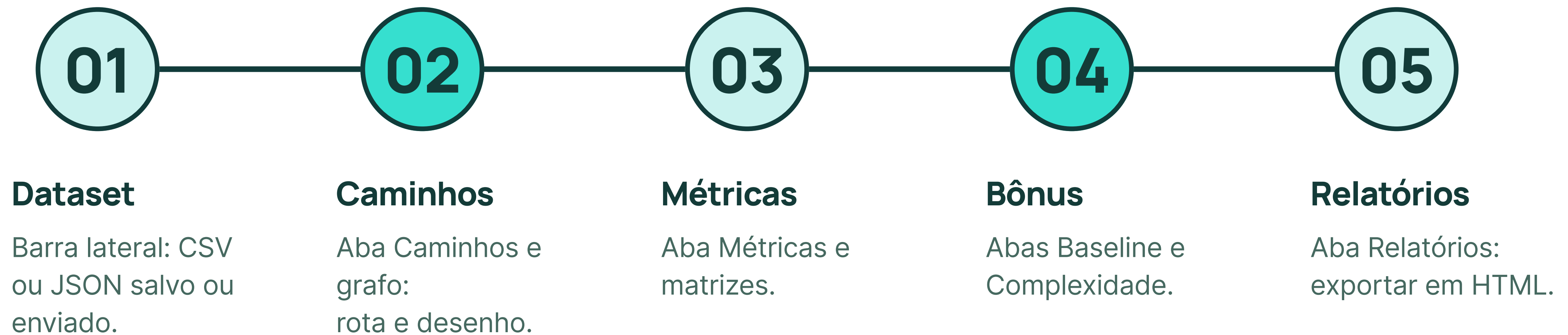


ESTOU EM	PRÓXIMO PASSO
A	B
B	C
C	D

**A → D: 10 direto,
6 via B e C.**

Comece em A e siga next até D: A → B → C → D. Exemplo de grafo_exemplo.csv.

Uma interface em Streamlit: carregue um grafo, rode a análise, explore as abas.



Carregar, executar, explorar.

Dataset

Dataset salvo

grafo_exemplo.csv

Ou envie um CSV/JSON

Upload

200MB per file • CSV, JSON

Salvar upload

CSV: source,target,weight. JSON aceita vertices e edges, incluindo vértices isolados.

Floyd-Warshall

Laboratório de caminhos mínimos • implementação própria • grafos dirigidos

grafo_exemplo.csv

Vértices

4

Arestas

4

Dados de entrada

Executar análise

Caminhos e grafo

Métricas e matrizes

Baseline

Complexidade

Relatórios

Origem

A

Destino

C

Distância mínima

5

A → B → C

■ caminho mínimo consultado

» demais arestas

Caminhos e grafo

Métricas e matrizes

Baseline

Complexidade

Relatórios

	vertices	edges	density	execution_time_ms	peak_memory_kb	reachable_pairs	unreachable_pairs	average_distance	minimum_distance	maximum_distance
0	4	4	33.3333	0.0256	1.4063	6	6	3.3333	1	6

Tempo em ms sem instrumentação. Pico em KiB de alocações Python, medido em outra execução; não é memória total do processo. Pares e densidade excluem a diagonal.

Matriz inicial

	A	B	D	C
A	0.0	3.0	10.0	∞
B	∞	0.0	∞	2.0
D	∞	∞	0.0	∞
C	∞	∞	1.0	0.0

Menores distâncias

	A	B	D	C
A	0.0	3.0	6.0	5.0

grafo_ciclo_negativo.json

Vértices

3

Arestas

3

Dados de entrada

Executar análise

Ciclo de peso negativo detectado envolvendo 'A'.

Dois algoritmos, a mesma resposta.

Baseline: Bellman-Ford executado a partir de cada origem.

REDE	RESULTADO
Redes de 30, 40, 60 e 100 vértices	Concordância em todos os pares (comparacao_redes.csv).
100 vértices e 430 arestas	9.900 pares; P001 → P100 custa 57.
Tempo na rede de 100 vértices	108,5 ms contra 110,0 ms do Bellman-Ford por origem.
Grafo com ciclo negativo	Detectado; a análise é rejeitada.

O tempo cresce como V^3 .

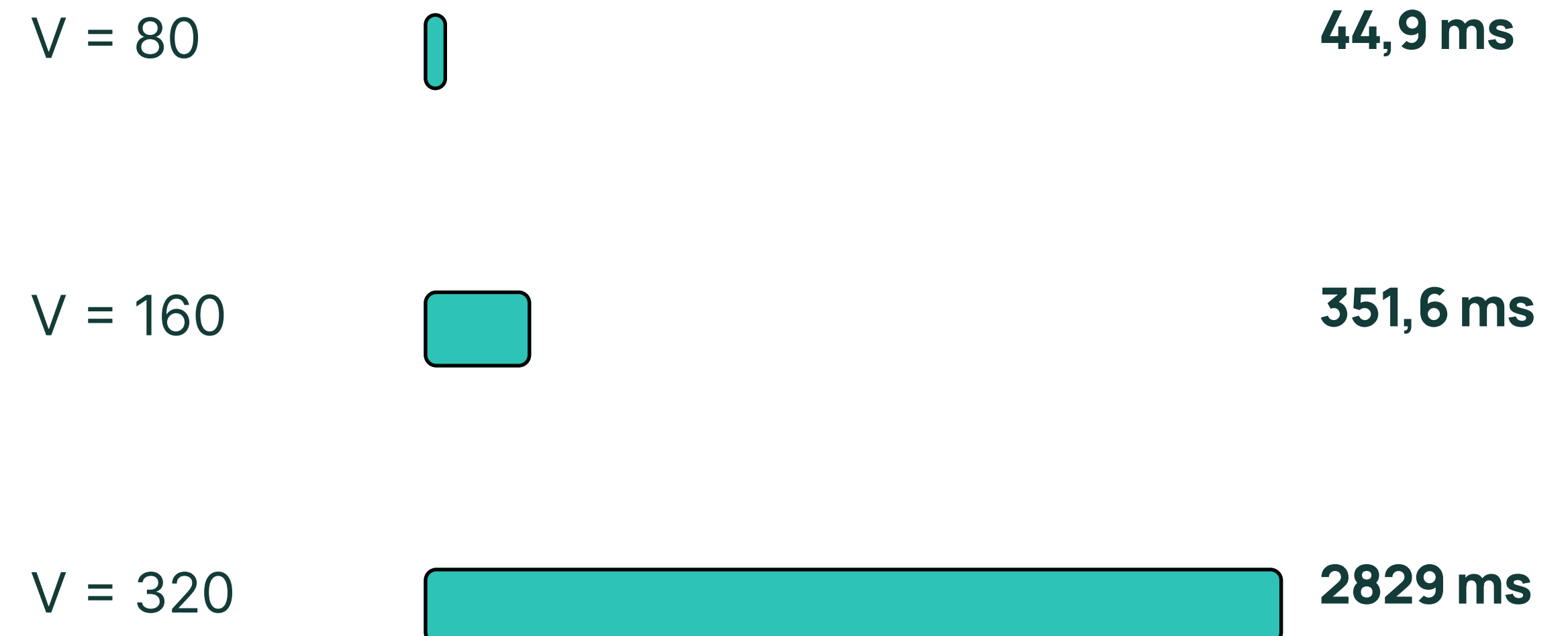
$O(V^3)$

Três laços sobre os V vértices.

Medido com V de 10 a 320.

A razão tempo/ V^3 fica quase constante.

TEMPO MEDIDO (MEDIANA DE 3 REPETIÇÕES)



Dobrar V multiplica o tempo por cerca de 8 (razão de 6,7 a 8,0); inclinação log-log tende a 3,0. Seis tamanhos não provam a lei assintótica.

Código pensado para ser mantido.

01

Arquitetura

domain, application, infrastructure, presentation.

02

→ SOLID

EvaluationService recebe um ShortestPathSolver; dois solvers, um contrato.

03

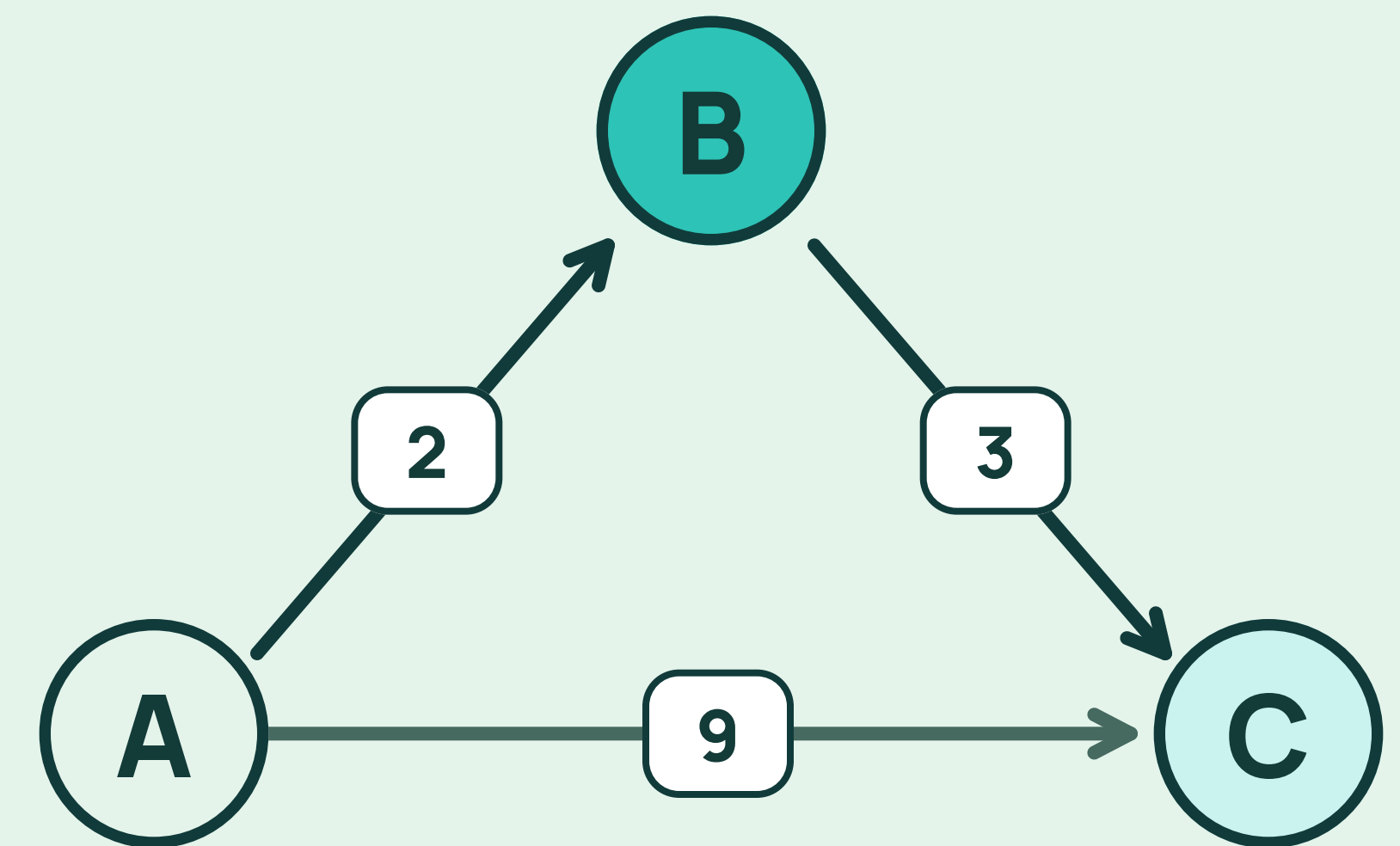
→ Qualidade

Poetry com lock, 51 testes, scripts regeneram reports/.

Obrigada.

Perguntas?

FLOYD-WARSHALL / Amanda Dias & Camilla Borchhardt



A → B → C → D